

Modelando problemas en PDDL

Javier Béjar, Javier Vázquez

2024/05/14

CS - GIA



El lenguaje PDDL

- ⊙ El Planning Domain Description Language (PDDL) es un lenguaje standard de descripción de planes
- ⊙ Desarrollado por la comunidad de investigadores en planificación para la competición mundial de planificadores <http://ipc.icaps-conference.org/>
- ⊙ En la actualidad se ha convertido en un estándar de facto, usado más allá de la competición

- ⊙ Según fueron pasando las ediciones, el nivel de dificultad de los problemas fue subiendo, y se crearon nuevas versiones del lenguaje para expresar esos problemas
- ⊙ Existen varias versiones de PDDL, desde la 1.0 a la 3.1, cada una de ellas con diferentes niveles de expresividad
 - La versión PDDL 1.0 se desarrolló para la 1ª edición de la competición, en 1998
 - La versión PDDL 2.1 es suficiente para todo lo que nosotros modelaremos en ABIA (se añadieron fluentes)
 - La última versión del lenguaje es PDDL 3.0
- ⊙ Hay que revisar la documentación (escasa) del planner que se está usando para saber que versión de PDDL soporta.
 - No existe ningún planificador que soporte la especificación PDDL 3.1 completa, sino subconjuntos de ella.

Describir dominios y problemas

Dominio + Problema

- ⊙ Un problema de planificación en PDDL se divide en dos conjuntos de definiciones:
 1. **Definición del dominio:** predicados, operadores, tipos, ...
 2. **Definición de una instancia del problema:** objetos, hechos del estado inicial y objetivo
- ⊙ Cada conjunto de definiciones está en un fichero separado.
 - Un único fichero de definición del dominio
 - Varios ficheros de definición de problemas (uno por cada instancia del problemas)

- ⊙ Requerimientos: Características del lenguaje PDDL que se usarán en la definición del problema
 - `:strips`, lenguaje básico
 - `:typing`, definición de tipos en parámetros y constantes
 - `:adl`, lenguaje avanzado con cuantificadores, efectos condicionales, ...
 - `:equality`, uso de comparaciones de igualdad
 - `:fluents`, lenguaje avanzado con funciones de coste y optimización
- ⊙ Tipos: Tipos de las constantes/parámetros que se usarán en el problema
 - facilitan la unificación entre variables del dominio y objetos de la instancia del problema
 - solo hacen falta cuando tenemos diferentes tipos de objetos que unificar
- ⊙ Predicados: Predicados que representan el problema
- ⊙ Acciones: Operadores de planificación para resolver el problema

```
(define (domain name-domain)
  (:requirements :adl :typing)
  (:types tipo1 tipo2 - object)

  (:predicates
    (pred1 ?V1 - tipo1)
    (pred2 ?V1 - tipo2 ?V2 - tipo1)
    (pred3)
  )

  (:action name-action1
    ...
  )
  (:action name-action2
    ...
  )
)
```

```
(:action name-action1
  :parameters (?V1 - tipo1 ?V2 - tipo2)
  :precondition (and
    (pred1 ?V1)
    (exists (?V3 - tipo2) (pred2 ?V3 ?V2))
    (not (pred3))
  )
  :effect (and (not (pred1 ?V1))
    (pred1 ?V2) (pred3)
  )
)
```

- ⊙ En la precondition
 - Un predicado atómico [strips]
 - Conjunción (and) de predicados atómicos [strips]
 - Combinación de condiciones con operadores lógicos (and/or/not) [adl]
 - Condiciones que solo aplican si otra condición es cierta (imply) [adl]
 - Fórmulas cuantificadas (forall,exists) [adl]
- ⊙ En los efectos
 - Un predicado atómico [strips]
 - La negación de un predicado atómico [strips]
 - La conjunción de predicados atómicos [strips]
 - Efectos que solo aplican si otra condición es cierta (when) [adl]
 - Efectos cuantificados universalmente (forall) [adl]

- ⊙ `:problem`, nombre de la instancia del problema
- ⊙ `:domain`, nombre del dominio del problema
 - ha de coincidir con el nombre del dominio en el fichero del dominio
- ⊙ `:objects`, son elementos relevantes del problema,
 - funcionan como valores constantes del lenguaje (que se unifican con las variables de los predicados del dominio)
- ⊙ `:init`, estado inicial del problema
 - es siempre una conjunción (aunque no se escribe hay un `and` implícito) de literales positivos (no se puede usar el `not`)
- ⊙ `:goal`, objetivo del problema
 - es una condición en la que se pueden usar muchos de los elementos del lenguaje (`and`, `or`, `not`, cuantificadores...)

10

```
(define (problem name-problem)
  (:domain name-domain)
  (:objects obj1T1 obj2T1 obj3T1 - tipo1
            obj1T2 obj2T2 obj3T2 - tipo2
  )
  (:init
    (pred1 obj1T1)
    (pred1 obj2T1)
    (pred1 obj3T1)
    (pred2 obj1T2 obj1T1)
    (pred2 obj2T2 obj2T1)
    (pred2 obj3T2 obj3T1)
  )
  (:goal (and (pred3) (forall (?V - tipo1) (not (pred1 ?V)))
  )
  )
)
```

11

- ⊙ `(imply (condition) (expression))`
 - solo en precondition y goal, sirve para tener (pre)condiciones condicionadas
 - si (y solo si) condition es cierta se verifica que la expression sea cierta
- ⊙ `(when (condition) (expression))`
 - solo en effect, sirve para tener efectos condicionados
 - si (y solo si) condition es cierta la expression se evalua, modificando predicados ciertos y falsos
- ⊙ `(exists (?x - type) (expression))`
 - solo en precondition
 - intenta ver si existe algun elemento x que cumpla la expression
- ⊙ `(forall (?x - type) (expression))`
 - en precondition, effect y goal
 - si está en precondition o goal, comprueba que la expression se cumple para todo x
 - si está en effect, evalua la expression para todo x

Mejoras en los modelos

- ⊙ No tenemos control sobre el código de los planificadores
- ⊙ la única forma de reducir el tiempo de cómputo de los planes es intentar ponerle las cosas más fácil al planificador
- ⊙ Algunas mejoras que se pueden hacer:
 1. uso de tipos y subtipos
 2. mejoras en la evaluación de condiciones
 3. añadir predicados filtro en la precondition
 4. añadir predicados guía en la precondition.
 5. no abusar del uso de forall y exists

14

Mejora 1 - Uso de tipos y subtipos

- ⊙ Para ganar eficiencia es importante facilitar la unificación de variables con objetos
- ⊙ Siempre que tengamos objetos de más de un tipo se deben definir y usar tipos, para reducir el número de objetos que el planificador intentará unificar con cada variable
- ⊙ Podemos aprovecharnos de la posibilidad de definir una jerarquía de tipos, creando subtipos para clasificar mejor los objetos y reducir aun más el número de posibles unificaciones

15

- ⊙ Para ganar eficiencia es importante el orden en el que se escriben las condiciones
- ⊙ Esto hay que tenerlo en cuenta en las condiciones de los operadores y en el objetivo
- ⊙ Lo habitual es poner primero las condiciones más restrictivas (las que unifican con menos objetos)
- ⊙ También es necesario evitar formulas que generen unificaciones demasiado complejas (por ejemplo con muchas disyunciones o existenciales)

- ⊙ Para ganar eficiencia podemos reducir aun más las posibles unificaciones de variables con objetos
- ⊙ Idea: añadir en la precondition de las acciones uno o más predicados filtro que reduzcan las posibles unificaciones variable-objeto
- ⊙ Ejemplos de predicados filtro:
 - Añadir al modelo un predicado (servido ?p) que indique que ?p ya cumple su parte del objetivo del problema. En las acciones añadimos la condición (not (servido ?p)) al principio para que el planificador no unifique con objetos que ya están servidos
 - Si el problema requiere que los objetos pasen por una serie de estados o pasos, crear predicados para saber en que estado/paso está cada objeto y restringir en la precondition de las acciones que los objetos unificados solo puedan estar en el estado/paso adecuado para cada acción

- ⊙ A veces añadir predicados filtro no es suficiente
- ⊙ Para acelerar la resolución del plan, podemos intentar guiar al planificador
- ⊙ Idea añadir predicados guía que indiquen al planificador que unificaciones hacer para que vaya más directo hacia la solución.
- ⊙ ALERTA: estamos añadiendo conocimiento sobre COMO resolver el problema en un modelo declarativo. Usar con moderación.

18

- ⊙ Ejemplos de predicados guía:
 - Restringir los objetos con los que se intenta unificar a un subconjunto controlado (por ejemplo, en vez de un swap de cualquier elemento a cualquier elemento en un vector, reducirlo a elementos en casillas adyacentes). Implica añadir predicados adicionales al modelo (por ejemplo `(adyacente ?x ?y)`) y añadirlo como restricción en la precondition de la(s) acción(es)
 - Añadir en el estado inicial predicados sobre parte de las condiciones del estado final (en el ejemplo de un vector: añadir un predicado `(target ?x ?p)`) que indica que `?x` debe acabar en la posición `?p`), y restringir las unificaciones de alguna(s) variable(s) de la acción a las que cumplen directamente esa condición del estado final (en el caso del vector, restringir el swap de forma que uno de los elementos que se mueve pase siempre a su posición definitiva).

19

- ⊙ Como hemos visto, uno de los puntos críticos en la eficiencia de la planificación automática es la exploración de todas las posibles unificaciones de objetos con variables en la precondition de las acciones.
- ⊙ los planificadores suelen poner mucho esfuerzo en optimizar estos procesos de unificación
- ⊙ normalmente el código que unifica variables de la precondition es mucho más eficiente que el código que unifica variables en un `exists` o en un `forall`.
- ⊙ Idea: no abusar del uso de `forall` y `exists`, usarlos con moderación

- ⊙ ¿Cuándo usar (o no) `forall` y `exists` en la precondition?:
 - Cuando se ha de comprobar en la precondition si una cierta condición se cumple para un conjunto de N objetos (N puede ser cero): deberemos usar el `forall`
 - Cuando hemos de comprobar si existe un objeto que cumpla una cierta condición (pero puede que no exista): debemos usar el `exists`
 - Cuando hemos de comprobar si existe un objeto que cumpla una cierta condición (y sabemos seguro que existe al menos uno): en vez de usar el `exists`, añadiremos una variable más como parámetro de la acción, y en la precondition añadiremos directamente la condición con esa variable extra (forzará al planificador a encontrar el objeto que unifica con la variable extra, tal y como haría el `exists`, pero de forma más eficiente)

Fluentes

- ⊙ La extensión `:fluents` permite extender el lenguaje para incluir:
 - Funciones (de hecho son variables numéricas)
 - Comparaciones aritméticas entre funciones
 - Operaciones aritméticas entre funciones
 - Añadir costes en los efectos de las acciones
 - Optimización del plan a partir de los valores de las funciones

- ⊙ Declarar en el apartado de requerimientos `:fluents`
- ⊙ Especificar una sección de funciones


```
(:functions
  (funcion1 ?V - tipo1)
  (funcion2 ?V - tipo2)
  (coste-total))
```
- ⊙ Los valores de las funciones se pueden consultar y comparar en las precondiciones de las acciones


```
(>= (+ (funcion1 ?V1) (funcion2 ?V1)) 10)
```
- ⊙ Se puede modificar el valor en los efectos de las acciones


```
(:effect (increase (coste-total) 1))
(:effect (decrease (funcion1 ?V1) (funcion2 ?V1)))
```

- ⊙ Inicializar los valores de los fluentes (para cada constante si es necesario)


```
(:init
  ...
  (= (funcion1 obj1T1) 0)
  (= (coste-total) 0)
  ...)
```
- ⊙ Incluir una sección que indica qué fuente optimizar (solo uno)


```
(:metric minimize (coste-total))
(:metric maximize (funcion1 obj1T1))
```
- ⊙ Se pueden optimizar múltiples fluentes usando una combinación lineal de sus valores


```
(:metric minimize
  (+ (coste1) (+ (* 5 (coste2)) (* 3 (coste3)))))
```